



**Đề tài: Giải thuật A* tìm kiếm đường đi trong hệ thống
ga tàu điện ngầm của thành phố Hồng Kông**

Giảng viên hướng dẫn: PGS.TS.Trần Đình Khang

Học phần: IT3160 – Nhập môn trí tuệ nhân tạo

Nhóm tác giả:

STT	Họ và tên	MSSV
1	Nguyễn Duy Anh	202416116
2	Cao Thành Đạt	202416152
3	Nguyễn Hữu Đức	202416159
4	Nguyễn Duy Hoàng	202416496
5	Đoàn Quốc Khánh	202416243
6	Đỗ Văn Nam	202416299

Hà Nội – Ngày 4 tháng 5 năm 2026



Giải thuật A* tìm kiếm đường đi trong hệ thống ga tàu điện ngầm của thành phố Hồng Kông

Học phần: IT3160 - Nhập môn Trí tuệ nhân tạo
Giảng viên hướng dẫn: PGS.TS.Trần Đình Khang

N. H. Đức ^{1,†}, C. T. Đạt ¹, Đ. Q. Khánh ¹,
Đ. V. Nam ¹, N. D. Hoàng ², N. D. Anh ³

¹Khoa học máy tính 05 K69* ²Khoa học máy tính 01 K69* ³Khoa học máy tính 04 K69*

*Trường Công nghệ Thông tin và Truyền Thông, Đại Học Bách Khoa Hà Nội

[†]Trung tâm nghiên cứu quốc tế về Trí tuệ nhân tạo, BKAI

Hà Nội, ngày 2 tháng 5 năm 2026

Mục lục

1. Giới thiệu chung
2. Mô tả và biểu diễn bài toán
3. Phương pháp tìm kiếm
4. Cài đặt chi tiết
5. Một số tính năng bổ sung
6. Kết luận

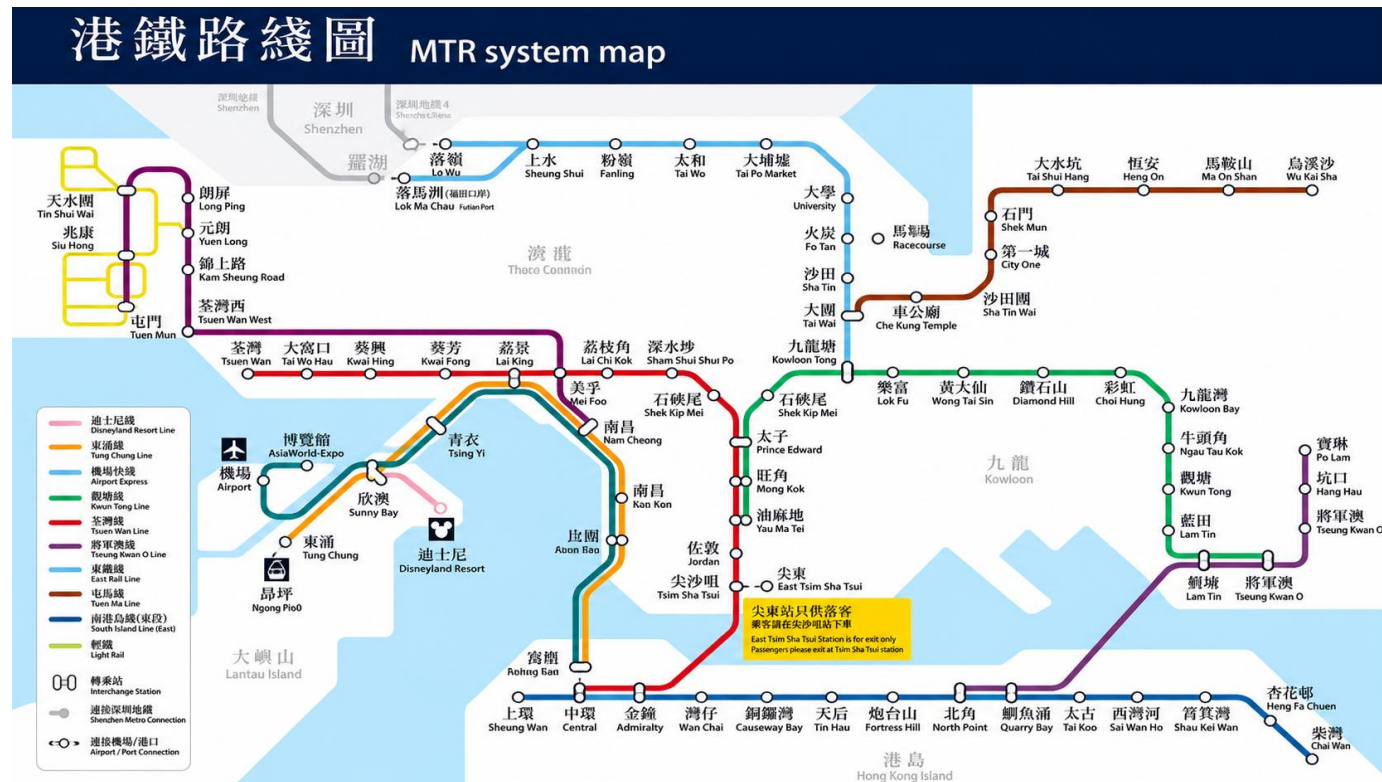
1. Giới thiệu chung

1.1. Hệ thống ga tàu điện ngầm Hồng Kông

1.2. Ứng dụng tìm đường

Hệ thống ga tàu điện ngầm Hồng Kông

- Hệ thống tàu điện ngầm MTR (Mass Transit Railway) tại Hong Kong là một trong những mạng lưới giao thông đô thị hiệu quả nhất thế giới, vận chuyển hơn 5 triệu hành khách mỗi ngày.



Hệ thống ga tàu điện ngầm Hồng Kông

- Vấn đề:

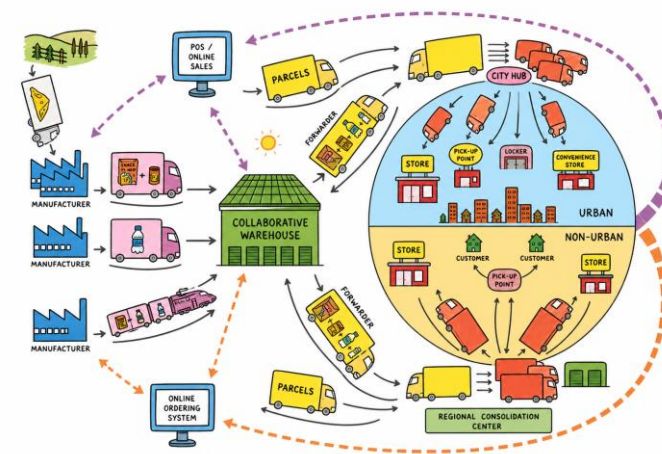
- Các sự cố kỹ thuật
- Bảo trì định kỳ
- Thảm họa tự nhiên
-

→ Gây gián đoạn trên một hoặc nhiều đoạn tuyến, buộc hành khách phải tìm tuyến đường thay thế trong thời gian ngắn.



Hệ thống ga tàu điện ngầm Hồng Kông

- Bài toán tìm đường tối ưu trên đồ thị có ràng buộc động (cạnh bị chặn thay đổi theo thời gian thực) là bài toán kinh điển trong lý thuyết đồ thị. Ứng dụng rộng rãi:
 - Hệ thống thông tin địa lý (GIS)
 - Lập kế hoạch logistics
 - Robot tự hành
 - ...
- Dự án MTR Navigator ra đời nhằm minh họa trực quan bài toán này trong một ngữ cảnh thực tiễn và gần gũi.



Mục tiêu

- ❖ Xây dựng ứng dụng web tương tác hiển thị mạng lưới ga MTR trên nền bản đồ địa lý thực tế
- ❖ Cài đặt thuật toán A^* để tìm đường ngắn nhất có tính đến các đoạn bị vô hiệu hoá
- ❖ Cho phép người dùng thao tác thời gian thực:
 - ✓ Thêm/xoá ràng buộc trực tiếp trên giao diện người dùng
 - ✓ Quan sát kết quả ngay lập tức
- ❖ Thiết kế giao diện trực quan, hiện đại theo chuẩn UI/UX chuyên nghiệp
- ❖ Phân tách rõ ràng giữa cấu trúc, giao diện và logic theo nguyên tắc Separation of Concerns

Ứng dụng tìm đường

- ✓ Ứng dụng web MTR Navigator - một hệ thống mô phỏng tìm đường tàu điện ngầm tương tác trên mạng lưới MTR tuyến Hong Kong
- ✓ Cho phép người dùng đánh dấu các đoạn đường gián đoạn, tự động tính toán tuyến đường tối ưu tránh các đoạn bị chặn
- ✓ Sử dụng thuật toán A* kết hợp với hàm heuristic địa lý Haversine
- ✓ Được xây dựng hoàn toàn bằng công nghệ web thuần (HTML5, CSS3, JavaScript ES6+) kết hợp thư viện bản đồ Leaflet.js, không phụ thuộc vào framework nặng nề
- ✓ Kiến trúc ba lớp tách biệt (Separation of Concerns) → đảm bảo tính dễ bảo trì và mở rộng

2. Mô tả và biểu diễn bài toán

2.1. Mô tả

2.2. Biểu diễn bài toán

Mô tả

- ❑ Như đã đề cập, nhiệm vụ là đi tìm tuyến tàu (metro) ngắn nhất có thể để kết nối giữa 2 địa điểm được chọn trên bản đồ
- ❑ Phiên bản hiện tại gồm 97 ga: Central, Admiralty, Wan Chai, Causeway Bay, Tin Hau, Fortress Hill, North Point,....
- ✓ Được giới hạn trong phạm vi trung tâm của thành phố, thuận tiện cho việc tìm đường

Biểu diễn bài toán

- Mạng lưới tàu điện ngầm được biểu diễn bằng đồ thị vô hướng có trọng số $G = (V, E, w)$, trong đó:
 - V : tập đỉnh, mỗi đỉnh đại diện cho một ga MTR với các thuộc tính:
 - ID
 - Tên hiển thị
 - Tọa độ địa lý (lat, lng)
 - E : tập cạnh, mỗi cạnh nối hai ga liền kề nhau trên tuyến
 - Do tàu MTR chạy hai chiều nên đồ thị là vô hướng
 - $w: E \rightarrow R^+$ là hàm trọng số: chi phí di chuyển qua một cạnh, hiện tại trong bài toán này là thời gian (phút)

Biểu diễn bài toán

- Danh sách kề (adjacency list) được chọn làm cấu trúc lưu trữ vì mạng lưới thưa:
 - Phù hợp với mạng metro nhiều tuyến, nhiều nhánh và có ga trung chuyển
 - Bộ nhớ $O(V + E)$
 - Trong repo hiện tại: $|V| = 97$ ga, $|E| = 106$ cạnh
 - Vì E xấp xỉ tuyến tính theo V nên có thể xem chi phí lưu trữ là $O(V)$ cho bài toán này

ID	Tên ga	Các ga kề	Toạ độ (lat, lng)
0	Central	1	22.2819, 114.1589
1	Admiralty	0, 2	22.2798, 114.1655
2	Wan Chai	1, 3	22.2770, 114.1731
3	Causeway Bay	2, 4	22.2803, 114.1856
4	Tin Hau	3, 5	22.2825, 114.1918
5	Fortress Hill	4, 6	22.2882, 114.1937
6	North Point	5	22.2913, 114.2006

Danh sách các ga

- Danh sách các ga và tọa độ của chúng được ghi chú chi tiết trong file *README.md*

Định nghĩa bài toán

- Bài toán được phát biểu hình thức như sau: Cho đồ thị $G = (V, E, w)$ và tập cạnh bị chặn $B \subseteq E$ (do người dùng định nghĩa), tìm đường đi có tổng trọng số nhỏ nhất từ đỉnh nguồn $s \in V$ đến đỉnh đích $t \in V$ trong đồ thị $G' = (V, E \setminus B, w)$. Nếu không tồn tại đường đi như vậy, trả về kết quả "không có đường"



3. Phương pháp tìm kiếm

3.1. Giải thuật sử dụng

3.2. Tổng kết thông số

Giải thuật sử dụng

- A* duy trì một danh sách mở (open list) chứa các node cần xét, ưu tiên mở rộng node có giá trị hàm đánh giá $f(n)$ nhỏ nhất

- Công thức cập nhật:

$$f(n) = g(n) + h(n)$$

- Trong đó:
 - $g(n)$: Chi phí thực tế đã đi từ node nguồn s đến node n (tổng trọng số các cạnh trên đường đi)
 - $h(n)$: Hàm heuristic — ước lượng chi phí còn lại từ n đến đích t (không vượt quá chi phí thực tế)
 - $f(n)$: Ước lượng tổng chi phí của đường đi tối ưu qua node n

Giải thuật sử dụng

- Thuật toán lặp lại:
 - Lấy node u có f nhỏ nhất từ open list, nếu $u = t$ thì truy vết đường đi và trả về kết quả
 - Ngược lại, duyệt tất cả hàng xóm v của u (bỏ qua các cạnh bị chặn), cập nhật $g(v)$ nếu tìm được đường ngắn hơn
 - Thêm v vào open list với $f(v) = g(v) + h(v)$
 - Quá trình kết thúc khi tìm được đích hoặc open list rỗng

Tại sao lại là A*?

Thuật toán	Ưu điểm	Nhược điểm	Phù hợp?
BFS	Đơn giản, tìm đường ngắn nhất (cạnh đơn vị)	Không xét trọng số cạnh	Không
Dijkstra	Tối ưu với trọng số dương	Duyệt tất cả node, chậm hơn A*	Được
A*	Tối ưu + nhanh hơn nhờ heuristic định hướng	Cần heuristic admissible phù hợp	Tốt nhất
Greedy	Rất nhanh	Không đảm bảo tối ưu	Không

Bảng 2: So sánh các thuật toán tìm đường

Lựa chọn hàm Heuristic h

- Sử dụng công thức Haversine

$$d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

Trong đó:

- ϕ_1, ϕ_2 : Vĩ độ của hai điểm (tính bằng radian).
 - $\Delta\phi, \Delta\lambda$: Độ chênh lệch vĩ độ và kinh độ.
 - r : Bán kính trung bình của Trái Đất (khoảng 6,371 km).
- Lý do:
 - Công thức Haversine là công thức **phù hợp trong thực tế nhất** để tính khoảng cách đường tròn lớn (đường chim bay) giữa hai điểm trên mặt cầu dựa trên vĩ độ và kinh độ
 - Đã được kiểm chứng và sử dụng rộng rãi trong đo đạc bản đồ

Lựa chọn hàm Heuristic h

- Công thức cài đặt thực tế

$$h(n) = \frac{\textit{harversine}(n, t)}{v_{MTR}}$$

- Trong đó $v_{MTR} = 400m/phút$ là tốc độ ước lượng của tàu MTR
- Đảm bảo $h(n) \leq$ chi phí thực tế (admissible), vì đường thẳng luôn ngắn hơn hoặc bằng đường trên đồ thị

Tổng kết thông số

Bảng thông số A* theo repo thực tế

Chỉ số	Giá trị	Ghi chú
Số đỉnh V	97	Tương ứng 97 ga MTR trong mảng stations.
Số cạnh E	106	Gồm 104 đoạn metro và 2 liên kết trung chuyển đi bộ.
Số tuyến	10	AEL, DRL, EAL, ISL, KTL, SIL, TCL, TKL, TML, TWL.
Số ga trung chuyển	21	Các ga có từ 2 tuyến trở lên trong thuộc tính lines.
Bậc lớn nhất	5	Bậc theo graph thực tế sau khi sinh từ lineSequences và transferLinks.
Số trạng thái A*	121	Repo tìm trên trạng thái stationId currentLine, không chỉ trên ga.
Độ phức tạp thời gian	$O(S^2 \log S + A)$	S là số trạng thái A*. Code hiện dùng open.sort() mỗi vòng, chưa dùng heap/priority queue.
Nếu dùng heap	$O(A \log S)$	Tối ưu hơn nếu thay open.sort() bằng priority queue.
Độ phức tạp không gian	$O(S + A)$	Dùng cho g, trace, closed, open và graph kề.
Worst case	Mở gần hết S trạng thái	Không còn là đồ thị tuyến tính 7 ga; khi nhiều đoạn bị cấm, A* có thể phải xét phần lớn graph.

4. Cài đặt chi tiết

4.1. Cấu hình sử dụng

4.2. Hướng dẫn chạy Repo

4.3. Bảo trì và nâng cấp mã nguồn

Cấu hình sử dụng

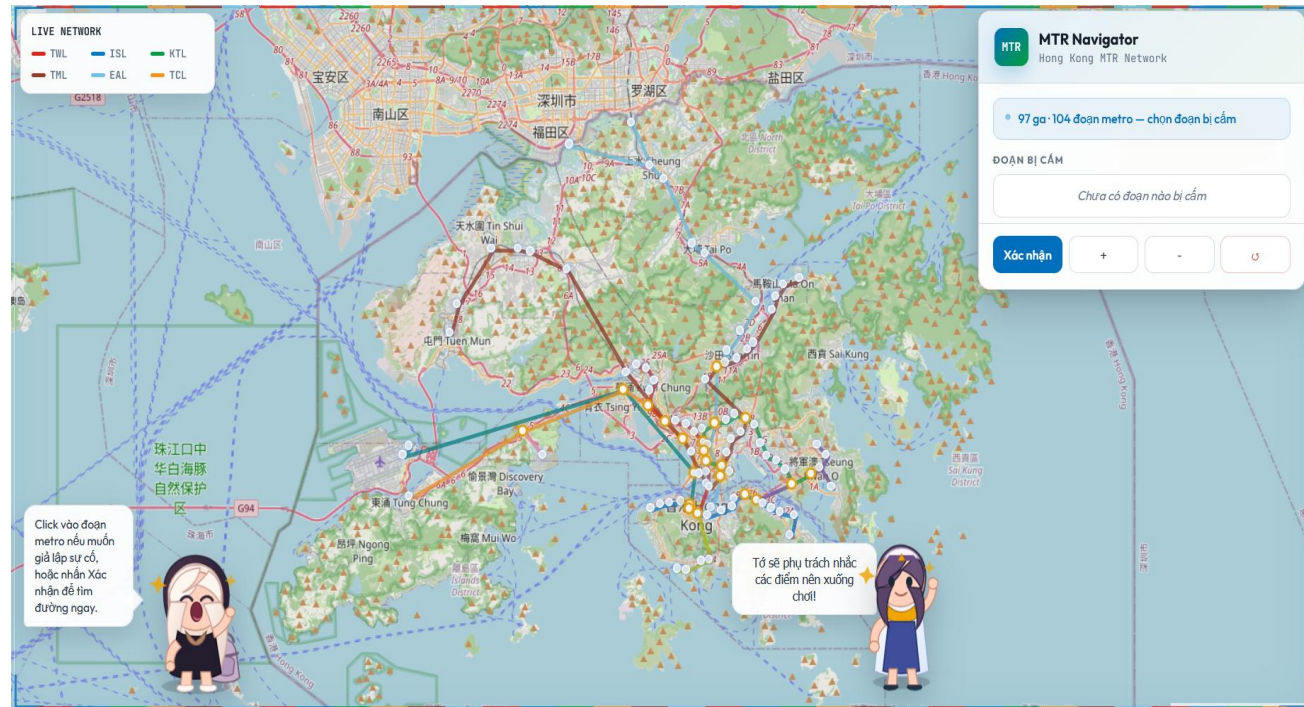
- Ngôn ngữ chính là JavaScript (Js) với các cài đặt phụ như sau:

Công nghệ / Công cụ	Phiên bản	Vai trò trong dự án
HTML5	Chuẩn W3C 2023	Cấu trúc DOM, semantic markup
CSS3	Chuẩn W3C	Giao diện, animation, CSS Custom Properties
JavaScript	ES6+	Logic ứng dụng, thuật toán, xử lý sự kiện
Leaflet.js	v1.9.x (CDN)	Hiển thị bản đồ tương tác, marker, polyline
OpenStreetMap	Tile API	Dữ liệu nền bản đồ địa lý mã nguồn mở
Google Fonts	CDN	Font Outfit (UI) và JetBrains Mono (code/data)

- Link Github dự án: <https://github.com/khanhdoan4106/mtrmap-test.git>

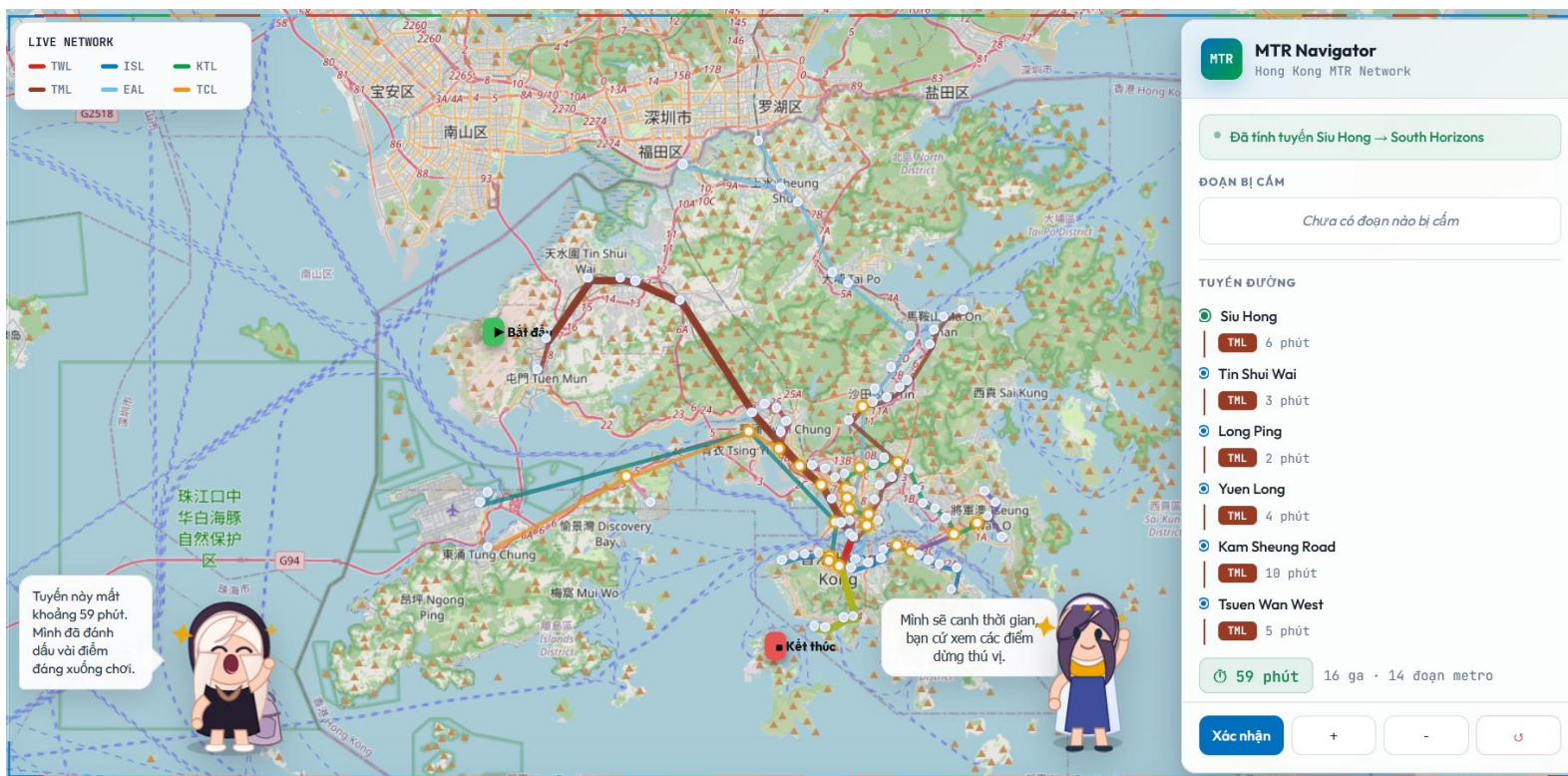
Hướng dẫn chạy repo

- Clone Repo về IDE cá nhân, chẳng hạn VSCode
- Tạo thư mục dự án NMAI_20252. Clone Repo về máy bằng lệnh:
 - ❖ git clone <https://github.com/khanhdoan4106/mtrmap-test.git>
- Tải Live Server trong Extension
- Click vào file .html và Open bằng Live Server
- Giao diện sẽ thu được như hình bên



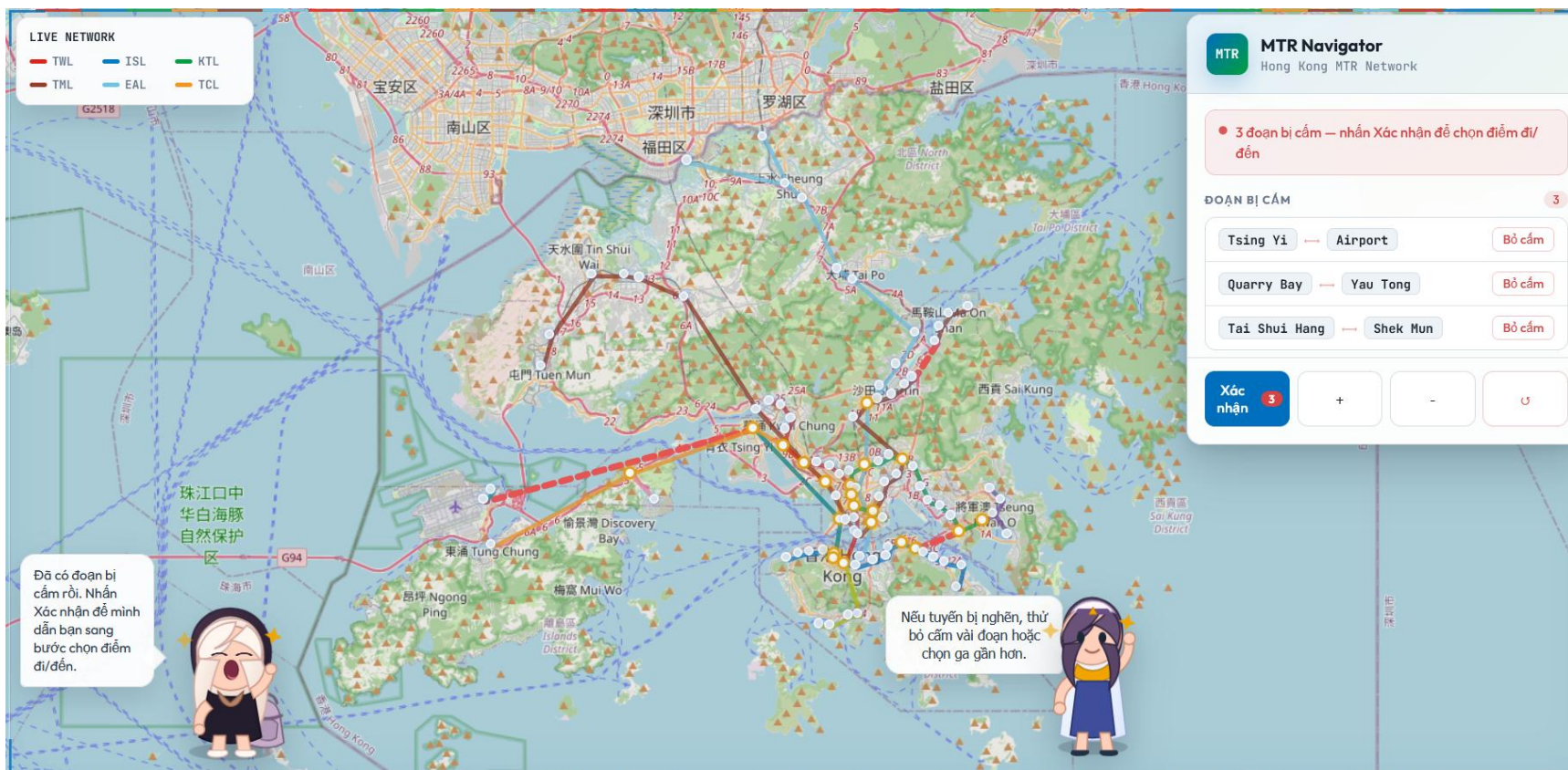
Hướng dẫn chạy repo

- Trong trường hợp không có đoạn nào bị cấm, bấm vào *Xác nhận*
- Sau đó, lựa chọn hai điểm bất kỳ trên bản đồ, ứng dụng sẽ trả về *tuyến metro có đường đi ngắn nhất và thời gian tới dự kiến*



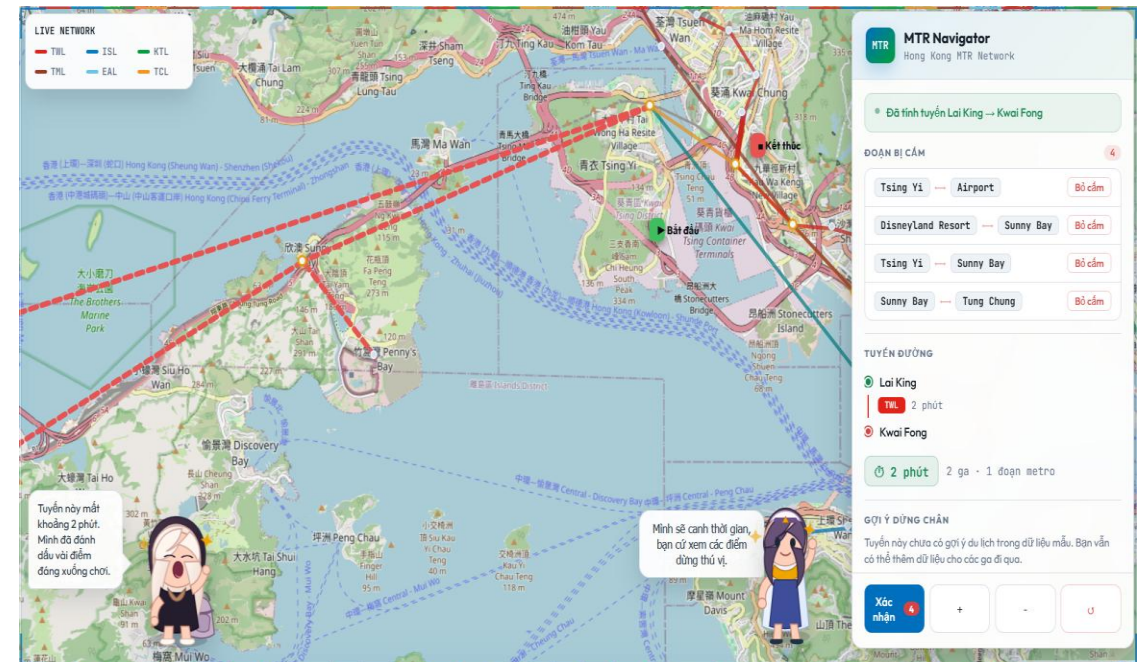
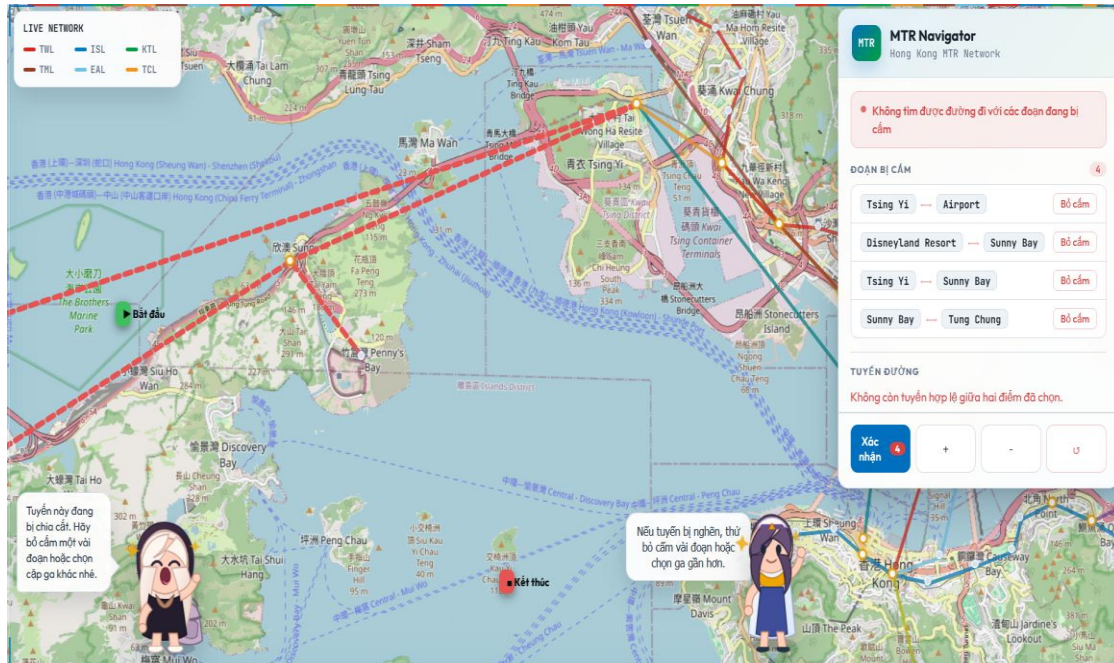
Hướng dẫn chạy repo

- Nếu muốn cấm đoạn nào, ngay từ lúc vào giao diện hãy chọn các tuyến đường cấm mong muốn
- Sau đó, vẫn tiếp tục hai lần chọn địa điểm đầu và cuối



Hướng dẫn chạy repo

- Nếu có tuyến metro phù hợp, hệ thống sẽ trả về *kết quả*. Nếu không, sẽ có thông báo *mọi tuyến đường đều bị chặn*
- Ngoài ra, thời gian tới dự kiến cũng được tính toán trước



Bảo trì và tinh chỉnh mã nguồn

- Mã nguồn được viết bằng Js hiện đại, cho phép xây dựng giao diện dễ dàng và đẹp mắt
- Thời gian giữa hai tuyến metro luôn được cập nhật theo thời gian thực khi di chuyển
- Có thể điều chỉnh màu sắc, thêm các tính năng mới cũng như cập nhật lại map theo thời gian bằng các tương tác API
- Mã nguồn được chỉnh sửa phải có comment cẩn thận, cách sử dụng cũng như phát triển cần được ghi lại chi tiết trong file *README.md*

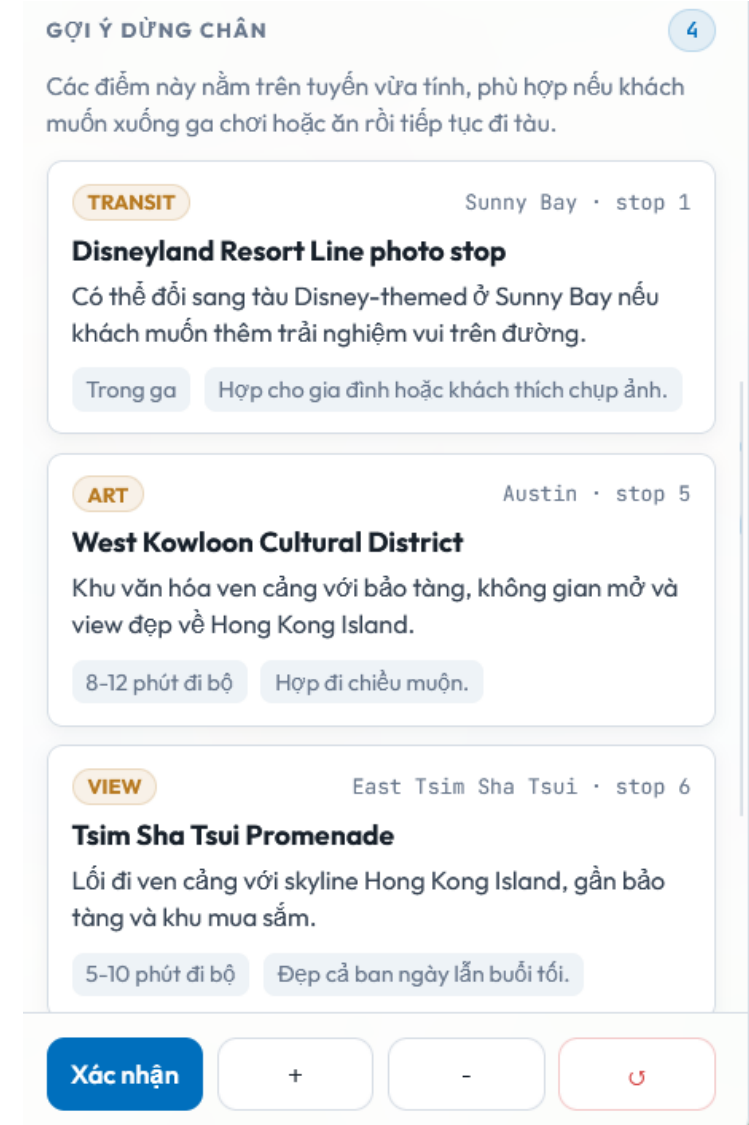
5. Một số tính năng bổ sung

- Ngoài các chức năng tìm kiếm cơ bản, dự án có tích hợp thêm hai “hướng dẫn viên” cực kỳ dễ thương ở hai góc
→ Mang lại cảm giác trải nghiệm người dùng thân thiện hơn
- Hai hướng dẫn viên tỷ hon sẽ minh họa lại thông tin giúp người dùng dễ theo dõi. Chả hạn:
 - Thời gian tới dự kiến
 - Các địa điểm tham quan
 - Các tuyến bị chặn
 - Các tuyến đường thay thế
 - ...



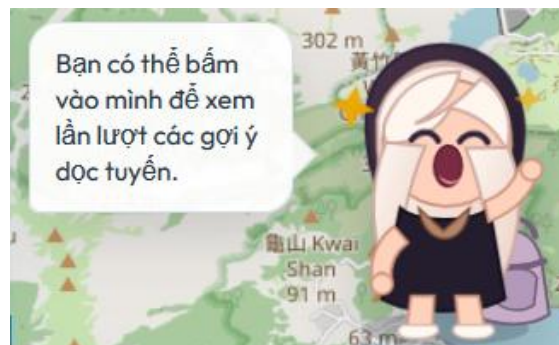
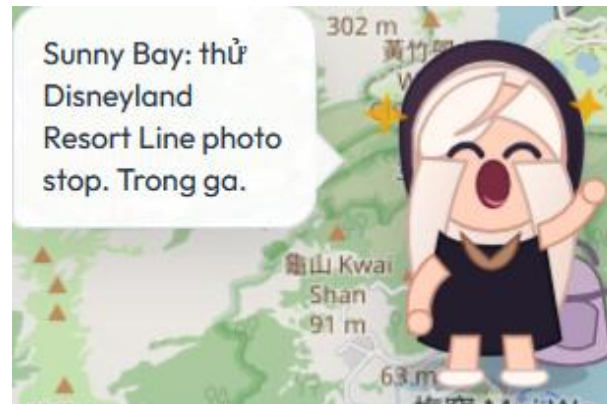
5. Một số tính năng bổ sung

- Ngoài ra dự án còn tích hợp thêm tính năng gợi ý địa điểm hay đồ ăn ngon cho người dùng, kèm theo đó là thời gian dự kiến tới địa điểm đó
- Gợi ý này kèm cả thời gian đi bộ dự kiến
- Có gợi ý đổi ga tàu nếu khách hàng muốn có thêm trải nghiệm trên hành trình metro của mình

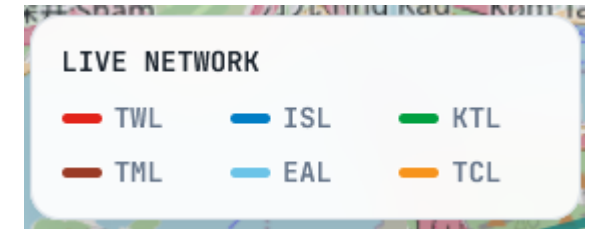
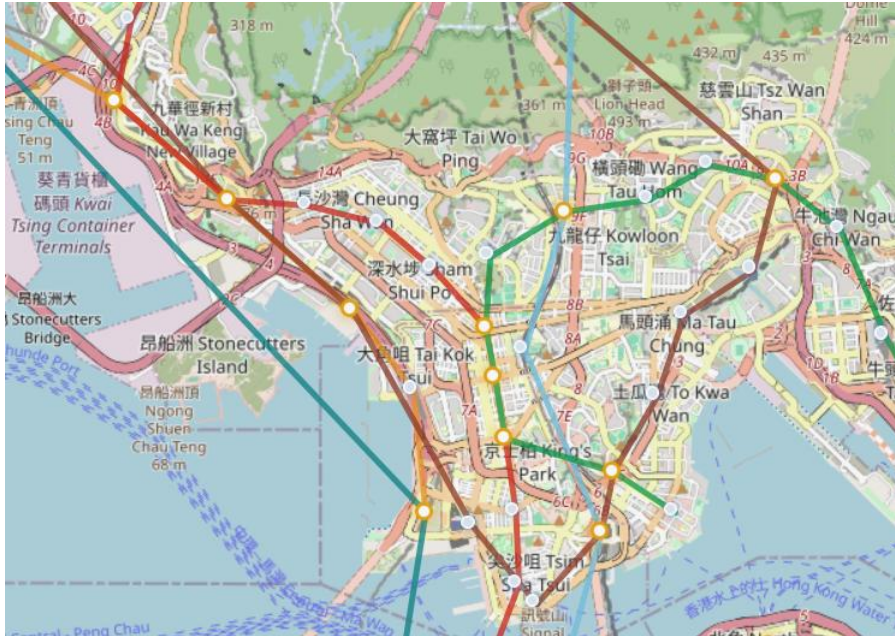


5. Một số tính năng bổ sung

- ❑ Sau khi người dùng chọn xong điểm đến và đi, Mascot tóc trắng sẽ thông tin cho khách hàng những thông tin hữu ích và thú vị; Mascot tóc tím sẽ nhấn mạnh lại những ý đó
- ❑ Đọc thông tin (hay tương tác) bằng cách bấm trực tiếp vào các nhân vật



Một vài hình ảnh trong giao diện web



6. Kết luận

- Project trình bày một công cụ tìm đường mới cho hệ thống Tàu điện ngầm của Hong Kong, với giao diện thân thiện, đẹp mắt và dễ thao tác
- Mọi ý tưởng trao đổi, đóng góp hay ý kiến vui lòng gửi về địa chỉ : duc.nh2416159@sis.hust.edu.vn
- Dự án sẽ còn được bảo trì, mở rộng và phát triển. Rất mong nhận được nhiều đóng góp quý báu từ thầy cô và các bạn
- Nghiêm cấm mọi hành vi sao chép. Nếu tái sử dụng mã của dự án vui lòng trích dẫn nguồn đầy đủ

Tài liệu tham khảo

- [1] DATA.GOV.HK - MTR routes, fares and barrier-free facilities
- [2] MTR Open Data - MTR Lines & Stations CSV
- [3] Wikidata Query Service
- [4] Leaflet
- [5] OpenStreetMap Copyright & License
- [6] Hong Kong Tourism Board - Discover Hong Kong

THANK YOU